Garbage collection

02 October 2026 18:51

Memory management in Javascript is performed automatically and invisibly to us. We create primitives, objects, functions… All that takes memory.

What happens when something is not needed anymore? How does the Javascript engine discover it and clean it up?

Reachability

The main concept of memory management in Javascript is reachability.

Simply put, reachable values are those that are accessible or usable somehow. They are guaranteed to be stored in memory.

1. Their base set of inherently reachable values that cannot be deleted for obvious reason.
   For instance:
   - The current executing function, its local variables and parameters.
   - Other functions on the current chain of nested call their local variables and parameters.
   - Global variables.
   - There are some other internal ones as well.

   These values are called roots.

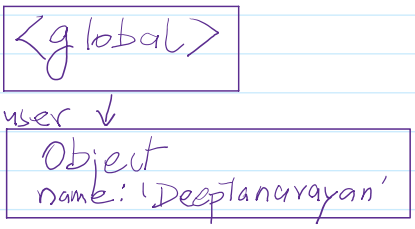
2. Any other value is considered reachable if its reachable from a route, by a reference or by a chain of reference.

   For instance, if there's an object in a global variable, and that object has a property referencing another object, that object is considered reachable. And those that if reference. Are also reachable. Detailed example to follow.

There's a background process in the Javascript engine that is called **garbage collector**. It monitors all objects and removes those that have become unreachable.

Here's the simplest example:

```
let user = {
  name:'Deeptanarayan'
};
```

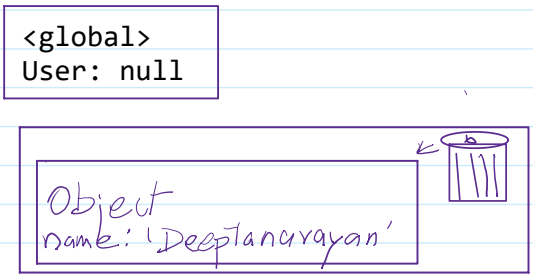


Here the arrow depicts an object reference. The global variable '*user*' reference the object {name: 'Deeptanarayan'.} (We will call it

'Deeptanarayan' for brevity)
The *'name'* property Of 'Deeptanarayan' stores a primitive, so it's painted inside the object.

If the value of user is overwritten, the reference is lost:

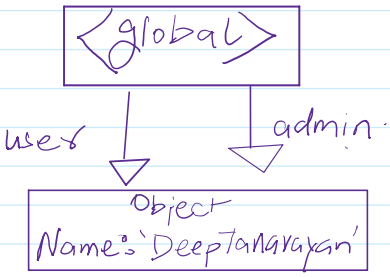
```
user = null;
```



Now Deeptanarayan becomes unreachable. There's no way to access it, no reference to it. Garbage collector will junk the data and free the memory.

Two reference
Now let's imagine we copied the reference from User to Admin.

```
let User = {
  Name:'Deeptanarayan';
};
let admin = User;
```



Now if we do the same.

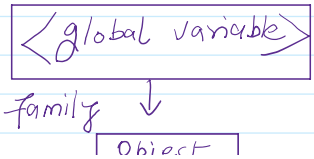
```
User = null;
```

… When the object is still reachable via admin global variable, so it must stay in memory. If we overwrite admin to then it can be removed.

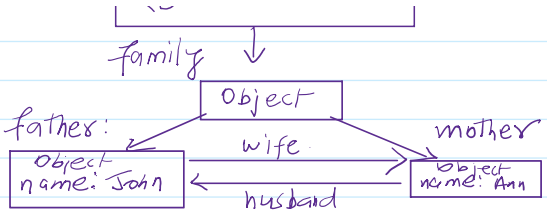
Interlinked object

Now a more complex example. The family:

```
function marry(man, women){
    women.husband = man;
    man.wife = women;
    return {
```



```
    women.husband = man;
    man.wife = women;
    return {
      father: man,
      mother: women
    }
}
let family = marry({
    name:'John'
},{
    name: 'Ann'
});
```

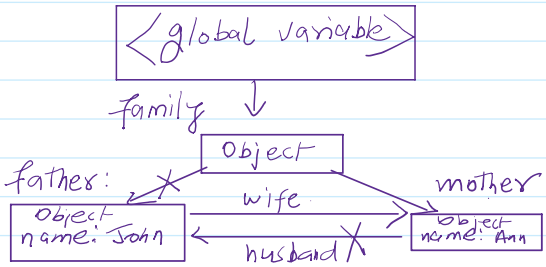


As of now all the objects are reachable

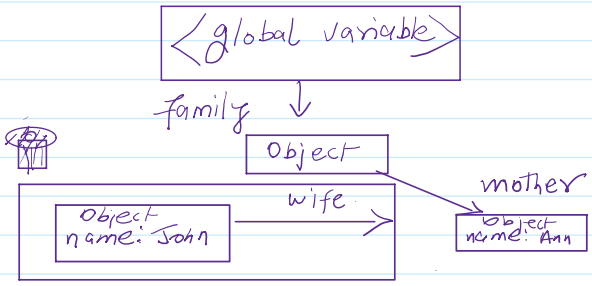
Function **mary** *"marries"* two objects by giving them references to each other and returns a new object that contains them both.

Now let's remove two references:

```
delete family.father;
delete family.mother.husband;
```

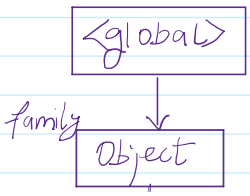


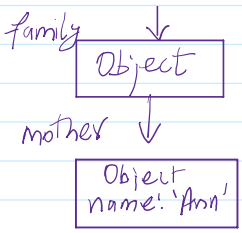
It's not enough to delete only one of these two references, because all objects would still be reachable. But if we delete both, then we can see that John has no incoming reference anymore.



Outgoing reference do not matter. Only incoming ones can make an object reachable. So John is now unreachable and will be removed from the memory with all its data that also become inaccessible.

After garbage collection:



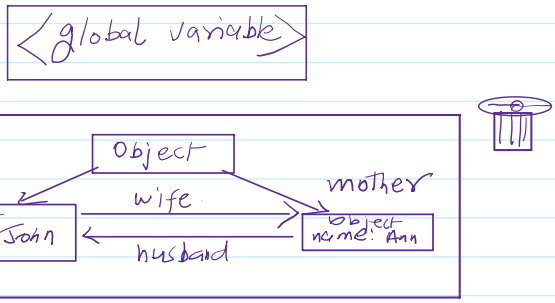


Unreachable island
It is possible that the whole island of interlinked objects become unreachable and is removed from the memory.

The source object is the same as above, then:

```
family = null;
```

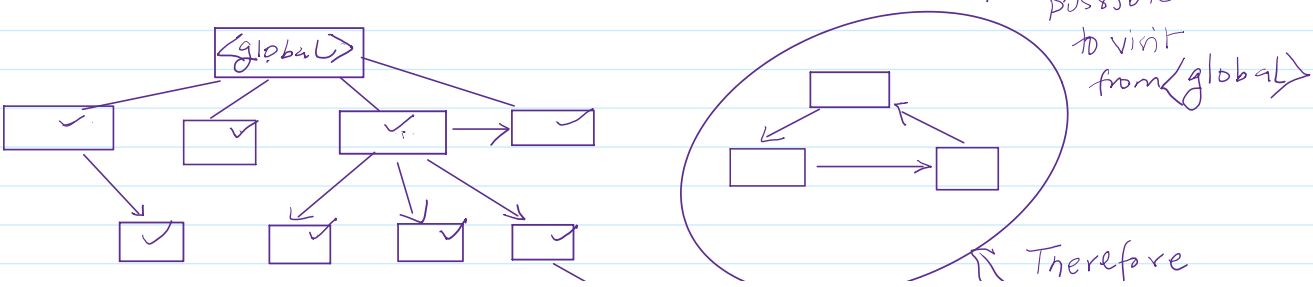
The in-memory picture becomes:

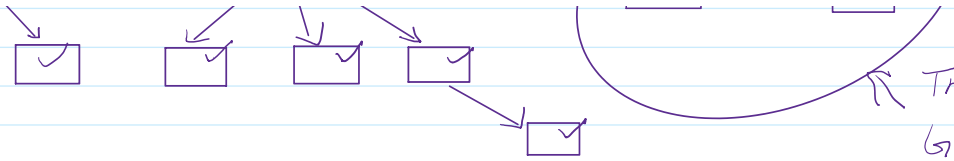


This example demonstrate how important the concept of reachability is. It's obvious that John and Anne are still linked. Both have incoming references, but that's not enough.
The former *'family'* object has been unlinked from the root, there's no reference to it anymore, so the whole island becomes unreachable and will be removed.

Internal algorithms
The basic garbage collection algorithm is called 'Mark-and-Sweep'.
The following *'garbage collection'* steps are regularly performed:

- The garbage collector takes root and "marks" (remembers) them.
- Then it visits and "marks" all references from them.
- Then it visits marked object and marks their references. All visited objects are remembered so as not to visit the same object twice in the future.
- … And so on until every reachable (from the roots) references are visited.
- All objects except marked ones are removed.





We can also imagine the process as spilling a huge bucket of paint from the root. That flows through all the references and marks all reachable objects. The unmarked ones are then removed.

That's the concept of how garbage collection works. Javascript engine apply many optimization to make it run faster and not introduce any delay into the code execution.

Some of the optimizations:

- **Generational collection:** Objects are split into two sets: "new ones" and "old ones". In typical code, many objects have a short life span: they appear, do their job and die fast. So it makes sense to track new objects and clear the memory from them if that's the case. Those that survive for long enough become "old" and re-examined less often.
- **Incremental collection -** If there are many objects and we try to walk and mark the whole object set at once, it may take some time and introduce visible delays in the execution. So the engine splits the whole set of existing objects into multiple parts. And then clear these parts one after another. There are many small garbage collection instead of a total one. That requires some extra bookkeeping between them to track changes, but we get many tiny delays instead of a big one.
- **Idle - time collection -** The garbage collector tries to run only while the CPU is idle to reduce the possible effect on the execution.

There exist other Optimizations and flavors of garbage collection algorithms. As much as I do like to describe them here, I have to hold off because different engines implement different tweaks and techniques. And what's even more important, things changes as Indian develop, so studying deeper in advance without a real need is probably not worth that.